Package javax.power.management

The Power Management package allows an application to monitor and respond to changes in the power states of the system.

See:

Description

Interface Summary	
PowerStateListener	An event listener interface for receiving PowerStateEvent.

Class Summary	
PowerManager	This abstract class is a common superclass which contains the methods utilized by normal applications to retrieve system power states information.
PowerState	This class defines the standard System Power States that are specified in the MNCRS Java Application Level Power Management Specification.
PowerStateEvent	Event object that is used for system power state change notifications.

Exception Summary	
PowerManagerException	Thrown when a power management related error is encountered.

Package javax.power.management Description

The Power Management package allows an application to monitor and respond to changes in the power states of the system. For example, the ON power state can be identified, and an application can respond to various conditions such as full power, power managed for efficiency, suspend, and sleep states.

Package Features

The Power Management API provides the following features:

- System power states be identified
- Application can receive notifications of Power state changes

Design Goals

The primary design goal for the Power Management API is to provide applications with accurate power state information so that it can respond appropritely as needed to save state or conclude transactions when state changes occur.

Package Architecture

When the PowerManager class is implemented, an application can use the `addPowerStateListener()` method in the `PowerManager` object to register to receive notifications. The application implements the `PowerStateListener`, an event listener interface, applications can identify system state activities such as Sleep, Suspend or PmActive. The current state can be retrieved from the `getPowerState()`method as illustrated in the following sample code.

Power Management Sample Code

```
/*
 * Copyright 1999 Sun Microsystems Inc, All rights reserved.
 */

import javax.power.management.*;

/**
 * Wait for and print changes in the power state.
 * Exit when the system is suspended or put to sleep
 * or 24 hours have passed.
 */
public class PowerSample2 implements PowerStateListener {

    // Get the system power manager.
    static PowerManager powermgr = PowerManager.getInstance();

    /**
     * Program to get and print the battery level and time remaining.
     */
    public static void main(String[] args) {
        PowerSample2 listener = new PowerSample2();

        // Register to be notified on power state changes
        powermgr.addPowerStateListener(listener);

        // Get the current time and battery level
        PowerState state = powermgr.getPowerState();

        System.out.println("Power state is " + state);

        try {                       // Wait up to a day
            Thread.sleep(24 * 60 * 60 * 1000);
        } catch (InterruptedException e) {
        }

        // Unregister for notifications
```

```
        powermgr.removePowerStateListener(listener);
    }

    /**
     * Gets notified of Power Manager state changes.
     */
    public void powerStateChange(PowerStateEvent event) {
        System.out.println("Changing power state from " +
                           event.getPrevState() +
                           " to " + event.getNewState());
        if (event.getNewState() == PowerState.Suspend ||
            event.getNewState() == PowerState.Sleep) {
            System.exit(0);
        }
    }
}
```

Hierarchy For Package javax.power.management

Package Hierarchies:

All Packages

Class Hierarchy

- class java.lang.Object
  - class java.util.EventObject (implements java.io.Serializable)
    - class javax.power.management.**PowerStateEvent**
  - class javax.power.management.**PowerManager**
  - class javax.power.management.**PowerState**
  - class java.lang.Throwable (implements java.io.Serializable)
    - class java.lang.Exception
      - class javax.power.management.**PowerManagerException**

Interface Hierarchy

- interface java.util.EventListener
  - interface javax.power.management.**PowerStateListener**

javax.power.management
Class PowerManager

```
java.lang.Object
  |
  +--javax.power.management.PowerManager
```

public abstract class **PowerManager**
extends java.lang.Object

This abstract class is a common superclass which contains the methods utilized by normal applications to retrieve system power states information. This class is designed as a singleton, there should only be one instantiation in a system.

The implementation of this class requires plaform specific knowledge. This class is intended for, and should be implemented by, system venders or developers for system-specific power manager implementations.

Normal applications should use the `getInstance` method to get the single instance of this object, and should call the class methods to make queries for power state information.

Method Summary

abstract void	**addPowerStateListener**(PowerStateListener listener) Used for adding a listener interested in power state events.
static PowerManager	**getInstance**() Returns a reference to the single instance of the PowerManger object.
abstract PowerState	**getPowerState**() Gets the current system power state.
abstract void	**removePowerStateListener**(PowerStateListener listener) Used for removing a listener no longer interested in power state events.
abstract void	**setPowerState**(PowerState state, boolean urgency) Puts the system in the state specified in the parameter "state".

Methods inherited from class java.lang.Object
equals, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Method Detail

getPowerState

```
public abstract PowerState getPowerState()
```

Gets the current system power state.

Returns:

the current system power state.

See Also:

PowerState

---

addPowerStateListener

```
public abstract void addPowerStateListener(PowerStateListener listener)
```

Used for adding a listener interested in power state events.

Parameters:

`listener` - the listener object to add.

See Also:

PowerStateEvent, PowerStateListener, removePowerStateListener(javax.power.management.PowerStateListener)

---

removePowerStateListener

```
public abstract void removePowerStateListener(PowerStateListener listener)
```

Used for removing a listener no longer interested in power state events.

Parameters:

`listener` - the listener object to remove.

See Also:

PowerStateEvent, PowerStateListener, addPowerStateListener(javax.power.management.PowerStateListener)

---

setPowerState

```
public abstract void setPowerState(PowerState state,
                                   boolean urgency)
                            throws PowerManagerException,
                                   java.lang.SecurityException
```

Puts the system in the state specified in the parameter "state".

Normal applications should not call this method.

The implementation of this method must perform the system dependent security check to guarantee that only privileged programs are allowed to access or initiate this method call. This is because this method will initiate system power state transitions. Failure to implement access control security protection and/or policy for this method may result in serious system damage.

Parameters:

`state` - the state that the system is going to transit to

`urgency` - If urgency is set to true, force the system to change state regardless of any objections which may be posted by applications to reject the state change. No queries will be sent to user during the state change process. In circumstances like a low power emergency shutdown, urgency should be set to true.

If urgency is set to false, applications may be able to oppose the state change event. The caller of the method may send out messages and/or user queries for notifications about the status of the call.

Throws:

PowerManagerException - A PowerManagerException must be thrown with error code STATE_TRANSITION_FAILURE if an error is encountered during the state change process. A PowerManagerException must be thrown with error code ILLEGAL_STATE_TRANSITION_REQUEST if an unrecognized "state" value is passed to the method or when there is an invalid state transition request.

java.lang.SecurityException - A java.lang.SecurityException must be thrown if the caller does not have the valid permission to initiate the call.

See Also:

`PowerStateEvent`, `PowerStateListener`, `PowerManagerException`

---

getInstance

```
public static PowerManager getInstance()
```

Returns a reference to the single instance of the PowerManger object.

Returns:

the instance of PowerManager Object, or `null` if power management is not implemented on this platform.

---

javax.power.management
Class PowerManagerException

```
java.lang.Object
  |
  +--java.lang.Throwable
        |
        +--java.lang.Exception
              |
              +--javax.power.management.PowerManagerException
```

public class **PowerManagerException**
extends java.lang.Exception

Thrown when a power management related error is encountered. Different errors can be identified by the error code. Method `getErrCode` can be used to retrieve the error that causes the exception.

Error code ILLEGAL_STATE_TRANSITION_REQUEST, STATE_TRANSITION_FAILURE, must be used by the subclass of `PowerManager` to indicate power state related errors.

Error code KEEP_CURRENT_STATE should be used by the `PowerStateEvent` listener when the listener wants to oppose the current state change event.

See Also:
`PowerManager`, `PowerStateEvent`, `PowerStateListener`, Serialized Form

Field Summary

static int	**ILLEGAL_STATE_TRANSITION_REQUEST** an illegal transition requested
static int	**KEEP_CURRENT_STATE** transition did not change states
static int	**STATE_TRANSITION_FAILURE** state transition failed

Constructor Summary

PowerManagerException(int err) Constructs a PowerManagerException object with an error code.
PowerManagerException(int err, java.lang.String msg) Constructs an PowerManagerException object with an error code and a detailed message.

Method Summary

int	**getErrCode**() Gets the exception error code.

Methods inherited from class java.lang.Throwable

fillInStackTrace, getLocalizedMessage, getMessage, printStackTrace, printStackTrace, printStackTrace, toString

Methods inherited from class java.lang.Object

equals, getClass, hashCode, notify, notifyAll, wait, wait, wait

Field Detail

ILLEGAL_STATE_TRANSITION_REQUEST

```
public static final int ILLEGAL_STATE_TRANSITION_REQUEST
```

an illegal transition requested

STATE_TRANSITION_FAILURE

```
public static final int STATE_TRANSITION_FAILURE
```

state transition failed

KEEP_CURRENT_STATE

```
public static final int KEEP_CURRENT_STATE
```

transition did not change states

Constructor Detail

PowerManagerException

```
public PowerManagerException(int err)
```

Constructs a PowerManagerException object with an error code.

Parameters:

`err` - the error code that is used to to create the exception object.

Throws:

java.lang.IllegalArgumentException - thrown for an unrecognized error code.

PowerManagerException

```
public PowerManagerException(int err,
                             java.lang.String msg)
```

Constructs an PowerManagerException object with an error code and a detailed message.

Parameters:

`err` - the error code that is used to create the exception object

`msg` - the detailed message

Throws:

java.lang.IllegalArgumentException - thrown for an unrecognized error code.

Method Detail

getErrCode

```
public int getErrCode()
```

Gets the exception error code.

Returns:

the error code that is used to construct the exception object.

---

javax.power.management
Class PowerState

```
java.lang.Object
  |
  +--javax.power.management.PowerState
```

public final class **PowerState**
extends java.lang.Object

This class defines the standard System Power States that are specified in the MNCRS Java Application Level Power Management Specification.

Each power state is represented by a static reference to a PowerState object.

Please refer to the MNCRS Java Application Level Power Management Specification for detail explanation on the power states.

Field Summary	
static PowerState	**FullPower** System is running at its maxium possible performance.
static PowerState	**Off** System is shutdown and is turned off.
static PowerState	**PmActive** System is operational but power consumption is regulated by power management software.
static PowerState	**Sleep** System is not operational, memory state is saved in low latency persistent storage that may require power.
static PowerState	**Suspend** System is not operational, memory state is saved in high latency persistent storage that do not require power.

Methods inherited from class java.lang.Object
equals, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Field Detail

FullPower

```
public static final PowerState FullPower
```

System is running at its maxium possible performance. Power management is not implemented or is disabled.

PmActive

```
public static final PowerState PmActive
```

System is operational but power consumption is regulated by power management software.

Sleep

```
public static final PowerState Sleep
```

System is not operational, memory state is saved in low latency persistent storage that may require power. Minimal power may be used but the system appears to be off. System can resume to its previous operational state in a relatively short period of time.

Suspend

```
public static final PowerState Suspend
```

System is not operational, memory state is saved in high latency persistent storage that do not require power. Power to the main system including CPU and memory are removed. System takes longer period of time to resume to its previous operational state.

Off

```
public static final PowerState Off
```

System is shutdown and is turned off. No memory state is saved. System needs to be rebooted to a operational state.

---

javax.power.management
Class PowerStateEvent

```
java.lang.Object
  |
  +--java.util.EventObject
        |
        +--javax.power.management.PowerStateEvent
```

public class **PowerStateEvent**
extends java.util.EventObject

Event object that is used for system power state change notifications. This event object should be created only by the system-specific power manager implementation. Normal applications should be the event listeners. Normal applications should use the `getNewState` and `getPrevState` methods to make queries for system power state information, and should use the `isUrgent` method to determine if the state change event can be opposed.

A PowerStateEvent can be initiated only when the operating system is operational. The following describes when a PowerStateEvent is valid and the valid state transitions.

A PowerStateEvent may be delivered before one of the following state transitions take place:

```
FullPower <-> PmActive
FullPower -> Sleep, Suspend, Off
PmActive -> Sleep, Suspend, Off
```

A PowerStateEvent may be delivered after one of the following state transition has taken place:

```
Sleep -> FullPower, PmActive
Suspend -> FullPower, PmActive
```

No PowerStateEvent can be delivered for the following state transition:

```
Sleep -> Suspend
```

See Also:
`PowerStateListener`, Serialized Form

Method Summary	
PowerState	**getNewState**() Gets the new power state that the system is going to transit to.
PowerState	**getPrevState**() Gets the previous power state, which is the state at the time when the event object is created.
boolean	**isUrgent**() Event listeners use this method to determine if the power state change event can be opposed.

Methods inherited from class java.util.EventObject
getSource, toString

Methods inherited from class java.lang.Object
equals, getClass, hashCode, notify, notifyAll, wait, wait, wait

Method Detail

getNewState

```
public PowerState getNewState()
```

Gets the new power state that the system is going to transit to.

Returns:

theNewState

---

getPrevState

```
public PowerState getPrevState()
```

Gets the previous power state, which is the state at the time when the event object is created.

Returns:

thePrevState

isUrgent

```
public boolean isUrgent()
```

Event listeners use this method to determine if the power state change event can be opposed.

If the return is true, the event source will ignore the PowerManagerException (with error code KEEP_CURRENT_STATE) thrown by the event listeners. State transition will occur. If the return is false, the event source will response to the PowerManagerException (with error code KEEP_CURRENT_STATE) thrown by the event listeners. The kind of response to the exception is system dependent.

Returns:

true or false

See Also:

`PowerStateListener`, `PowerManagerException`

javax.power.management
Interface PowerStateListener

public interface **PowerStateListener**
extends java.util.EventListener

An event listener interface for receiving PowerStateEvent.

Normal applications that require software preparations before the system transits from the FullPower or PmActive state to Sleep, Suspend or Off state, should implement this interface. Any normal applications that are interested in the system state transition activities may also implement this interface.

The application should create a listener object and use the `addPowerStateListener` method in the `PowerManager` object to register with the `PowerStateEvent` source. When a `PowerStateEvent` occurs, the `powerStateChange` method in the listener object is invoked, the `PowerStateEvent` object is passed to the listener, and the listener can get information about the event and respond to the event.

See Also:
`PowerStateEvent`, `PowerManager`, `PowerManagerException`

Method Summary

void	**`powerStateChange`**(`PowerStateEvent evt`) Invoked when the system is going to change state, or has changed state.

Method Detail

powerStateChange

```
public void powerStateChange(PowerStateEvent evt)
                     throws PowerManagerException
```

Invoked when the system is going to change state, or has changed state.

Parameters:
`evt` - the event object SystemStateChangeEvent

Throws:
PowerManagerException - A PowerManagerException with error code KEEP_CURRENT_STATE may be thrown by the listener if the listener wants to oppose the current state change event. The listener should use the `isUgrent` method in the `PowerStateEvent` object to examine if the state change event can be opposed. If the return

value from the `isUrgent` method call is true, the PowerManagerException thrown by the listener will be ignored by the event source; otherwise, the exception will be serviced by the event source. The response to the exception is system dependent.

See Also:

`PowerStateEvent`, `PowerManagerException`

Package javax.power.monitor

The Power Monitor package allows an application to monitor and respond to changes in the power level of the system.

See:

Description

Interface Summary	
PowerMonitorListener	An event listener interface for receiving events related to the PowerMonitor.

Class Summary	
PowerMonitor	A class for presenting power level status to applications.
PowerWarningType	A class that indexes the types of PowerWarnings an application can listen for.

Package javax.power.monitor Description

The Power Monitor package allows an application to monitor and respond to changes in the power level of the system. An application is able to retrieve the current battery level, an estimate of remaining battery life, and whether an external power source is being used. This package also enables an application to be notified when the battery power is running low and a service is about to be terminated as a result.

Package Features

The Power Monitor API provides the following features:

- Monitors available power levels in the device such as when a battery is running low
- Notifies an application when the power system is about to terminate due to low resources
- Application can access information on current battery level, estimate of remaining battery life, and whether external power source is being used
- Multiple standard warning types for wide range of devices available such as:
  - Battery level critically low
  - Returned to normal
  - Telephone connection is about to terminate due to insufficient power
  - External power source has been applied or removed
  - Infrared connection is about to terminate due to insufficient power
  - Network connection is about to terminate due to insufficient power

Design Goals

The primary design goal for the Power Monitor API is to provide applications with accurate power system activity so that a device does not prematurely terminate due to lack of power resources.

Package Architecture

Using the `PowerMonitorListener,` an event listener interface, the Power Monitor API can collect the power level status for applications. If the `powerWarning()` method is called by the power system, various warning types can be retrieved depending upon the current power status.

When the `getEstimatedSecondsRemaining()` and `getBattery Level()` methods are implemented by the Power Monitor class, an application can retrieve the current time and battery level for the device. This is further illustrated in the following sample code.

Power Monitor Sample Code

```
/*
 * Copyright 1999 Sun Microsystems Inc, All rights reserved.
 */

import javax.power.monitor.*;

/**
 * Print the battery level and time remaining..
 */
public class PowerSample1 {

    /**
     * Program to get and print the battery level and time remaining.
     */
    public static void main(String[] args) {

        // Get the system power monitor.
        PowerMonitor monitor = PowerMonitor.getInstance();

        // Get the current time and battery level.
        int remaining = monitor.getEstimatedSecondsRemaining();
        int level = monitor.getBatteryLevel();

        System.out.println("Battery Level = " + level +
                           ", " + remaining + " seconds remaining.");
        if (remaining < 60) {
            System.out.println("Battery almost dead, better save your work.");
        }
    }
}
```

Hierarchy For Package javax.power.monitor

Package Hierarchies:

All Packages

Class Hierarchy

- class java.lang.Object
  - class javax.power.monitor.**PowerMonitor**
  - class javax.power.monitor.**PowerWarningType**

Interface Hierarchy

- interface java.util.EventListener
  - interface javax.power.monitor.**PowerMonitorListener**

javax.power.monitor
Class PowerMonitor

```
java.lang.Object
  |
  +--javax.power.monitor.PowerMonitor
```

public class **PowerMonitor**
extends java.lang.Object

A class for presenting power level status to applications. The class is designed as a singleton, so there should only be one instantiation per active JVM. The class is designed to allow an application to retrieve the current battery level, an estimate of remaining battery life, and whether an external power source is being used. It also provides notifications that an application can register for that get called when battery power is running low and a service is about to be terminated as a result.

Method Summary

void	**addPowerMonitorListener**(PowerMonitorListener listener) Used for adding a listener interested in all of the power monitor warnings and events defined in the PowerWarningType class.
void	**addPowerMonitorListener**(PowerMonitorListener listener, PowerWarningType warningType) Used for adding a listener interested in specific types of power monitor warnings and events.
int	**getBatteryLevel**() Used for retrieving the current level (percent of battery charge remaining) of the main battery or batteries.
int	**getEstimatedSecondsRemaining**() Used for getting an estimate of how much time remains before battery power ceases.
static PowerMonitor	**getInstance**() Returns a reference to the single instance of the PowerMonitor object.
void	**removePowerMonitorListener**(PowerMonitorListener listener) Used for removing a listener from all power monitor warnings and events.
void	**removePowerMonitorListener**(PowerMonitorListener listener, PowerWarningType warningType) Used for removing a listener no longer interested in the specified power monitor warning or event.
boolean	**usingExternalPowerSource**() Used for checking if an external power source is being used.

Methods inherited from class java.lang.Object
`equals, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait`

Method Detail

getInstance

`public static PowerMonitor` **`getInstance`**`()`

Returns a reference to the single instance of the PowerMonitor object.

Returns:

Returns a reference to the PowerMonitor, or `null` if PowerMonitor is not implemented on this platform.

---

getBatteryLevel

`public int` **`getBatteryLevel`**`()`

Used for retrieving the current level (percent of battery charge remaining) of the main battery or batteries.

Returns:

Returns the amount of charge (capacity) remaining in the main battery or batteries, as an integer percentage (in the range of 0-100), or -1 if battery capacity information is not available, or Integer.MAX_VALUE if battery capacity information is not available and an external power source is currently being used.

---

getEstimatedSecondsRemaining

`public int` **`getEstimatedSecondsRemaining`**`()`

Used for getting an estimate of how much time remains before battery power ceases.

Returns:

Returns an estimate of the number of seconds of battery life remaining at the current rate of consumption, or -1 if no estimate is available, or Integer.MAX_VALUE if an external power source is currently applied.

usingExternalPowerSource

```
public boolean usingExternalPowerSource()
```

Used for checking if an external power source is being used.

Returns:

Returns true if an external power source is being used (implying infinite power remaining). Returns false if running from internal batteries, in which case the battery level and estimated seconds remaining should be monitored.

addPowerMonitorListener

```
public void addPowerMonitorListener(PowerMonitorListener listener,
                                    PowerWarningType warningType)
```

Used for adding a listener interested in specific types of power monitor warnings and events. If any power status condition that would cause a notification exists at the time a new listener is registered the notification must be delivered immediately to the new listener. If the listener is already registered for the specified warning type this method then no action will be taken.

Parameters:

`listener` - the PowerMonitorListener object to add.
`warningType` - the type of warning to being registered.

See Also:

`PowerMonitorListener.powerWarning(int, javax.power.monitor.PowerWarningType)`, `PowerWarningType`

removePowerMonitorListener

```
public void removePowerMonitorListener(PowerMonitorListener listener,
                                       PowerWarningType warningType)
```

Used for removing a listener no longer interested in the specified power monitor warning or event. If the listener is not registered for the specified warning type then no action is taken.

Parameters:

`listener` - the PowerMonitorListener object to remove.
`warningType` - the type of warning to being registered.

See Also:

`PowerMonitorListener.powerWarning(int, javax.power.monitor.PowerWarningType)`, `PowerWarningType`

addPowerMonitorListener

```
public void addPowerMonitorListener(PowerMonitorListener listener)
```

Used for adding a listener interested in all of the power monitor warnings and events defined in the PowerWarningType class. The listener will be registered for all PowerWarningTypes. If any power status condition that would cause a notification exists at the time a new listener is registered the notification must be delivered immediately to the new listener. If the listener is already registered this method will not register the listener again and the listener will receive all power warning types.

Parameters:

`listener` - the PowerMonitorListener object to add.

See Also:

`PowerMonitorListener.powerWarning(int, javax.power.monitor.PowerWarningType)`, `PowerWarningType`

---

removePowerMonitorListener

```
public void removePowerMonitorListener(PowerMonitorListener listener)
```

Used for removing a listener from all power monitor warnings and events. If the listener is not registered then no action is taken.

Parameters:

`listener` - the PowerMonitorListener object to remove.

See Also:

`PowerMonitorListener.powerWarning(int, javax.power.monitor.PowerWarningType)`, `PowerWarningType`

---

javax.power.monitor
Interface PowerMonitorListener

public interface **PowerMonitorListener**
extends java.util.EventListener

An event listener interface for receiving events related to the PowerMonitor.

An application can register for a number of different warning types, as represented in the PowerWarningType class.

See Also:
`PowerMonitor.addPowerMonitorListener(javax.power.monitor.PowerMonitorListener, javax.power.monitor.PowerWarningType)`, `PowerWarningType`

Method Summary

void	**`powerWarning`**`(int estimatedSecondsRemaining, PowerWarningType warningType)` This method is called by the power system when power events occur such as when the battery level is running low, and an application may need to start conserving power or alter its behavior as a result.

Method Detail

powerWarning

```
public void powerWarning(int estimatedSecondsRemaining,
                         PowerWarningType warningType)
```

This method is called by the power system when power events occur such as when the battery level is running low, and an application may need to start conserving power or alter its behavior as a result. There are a number of warning types defined in the PowerWarningType class.

Parameters:

`estimatedSecondsRemaining` - an estimate of the number of seconds remaining before the service ceases, or -1 if no estimate is available.

`warningType` - the type of warning/event being notified about.

See Also:

`PowerWarningType`, `PowerMonitor.getEstimatedSecondsRemaining()`

javax.power.monitor
Class PowerWarningType

```
java.lang.Object
  |
  +--javax.power.monitor.PowerWarningType
```

public final class **PowerWarningType**
extends java.lang.Object

A class that indexes the types of PowerWarnings an application can listen for. The intent is for this class to define the *standard* set of warnings across a range of devices.

Each field is a static reference to a PowerWarningType object and would be initialized as:

```
public static final PowerWarningType BatteryLow = new PowerWarningType();
```

See Also:
PowerMonitor.addPowerMonitorListener(javax.power.monitor.PowerMonitorListener, javax.power.monitor.PowerWarningType)

Field Summary

static PowerWarningType	**BatteryCritical** A warning that the primary battery level is critically low.
static PowerWarningType	**BatteryLow** A warning that the primary battery level is getting low.
static PowerWarningType	**BatteryNormal** An indication that the level of the primary battery has returned to normal (is no longer low or critical), presumably due to being charged.
static PowerWarningType	**CallTermination** A warning that the telephone connection is about to be terminated due to insufficient power, so the application should close down the session in an orderly fashion.
static PowerWarningType	**ExternalPowerSourceChange** An indication that an external power source has been applied or removed.
static PowerWarningType	**IrTermination** A warning that the infrared connection is about to be terminated due to insufficient power, so the application should close down infrared sessions in an orderly fashion.
static PowerWarningType	**NetworkTermination** A warning that the network connection is about to be terminated due to insufficient power, so the application should close down the network sessions in an orderly fashion.

Method Summary

java.lang.String	**toString**() Return a String describing this type of warning.

Methods inherited from class java.lang.Object

equals, getClass, hashCode, notify, notifyAll, wait, wait, wait

Field Detail

BatteryNormal

```
public static final PowerWarningType BatteryNormal
```

An indication that the level of the primary battery has returned to normal (is no longer low or critical), presumably due to being charged.

BatteryLow

```
public static final PowerWarningType BatteryLow
```

A warning that the primary battery level is getting low. The definition of a *low battery* is implementation dependent, but is a general indication that the application should begin conserving power.

BatteryCritical

```
public static final PowerWarningType BatteryCritical
```

A warning that the primary battery level is critically low. The application should immediately clean up and be prepared to shut down if necessary when it receives this warning. The device can be shutdown at any time with further warning.

NetworkTermination

```
public static final PowerWarningType NetworkTermination
```

A warning that the network connection is about to be terminated due to insufficient power, so the application should close down the network sessions in an orderly fashion.

CallTermination

```
public static final PowerWarningType CallTermination
```

A warning that the telephone connection is about to be terminated due to insufficient power, so the application should close down the session in an orderly fashion.

IrTermination

```
public static final PowerWarningType IrTermination
```

A warning that the infrared connection is about to be terminated due to insufficient power, so the application should close down infrared sessions in an orderly fashion.

ExternalPowerSourceChange

```
public static final PowerWarningType ExternalPowerSourceChange
```

An indication that an external power source has been applied or removed. The estimatedTimeRemaining in this case will be either Integer.MAX_VALUE if a power source was applied, a positive integer indicating an estimate of the time remaining if the power source has been removed, or -1 if the power source has been removed and no time estimate is available.

Method Detail

toString

```
public java.lang.String toString()
```

Return a String describing this type of warning.

Overrides:

toString in class java.lang.Object

Returns:

a descriptive string.

Package javax.power.management

Table of Contents